

Total: 10 points for 6 questions, Due at **11:59PM November 22, 2025**

What to submit: Put your answers in solution sections bellow and submit a PDF to Cat-Courses. Select all choices that apply for multiple-choices problems.

Student Name: **Po-Yun Cheng (276296347)**

1. (1 point) Calculate the probability of the sentence *i want to eat lunch*, given the probabilities for a bi-gram language model in Fig. 1. Assume $P(i|\langle s \rangle) = 0.19$ with start-symbol $\langle s \rangle$ and $P(\langle /s \rangle | \text{lunch}) = 0.40$ with end-symbol $\langle /s \rangle$.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 1: Bigram probabilities for eight words learned from a corpus. Zero probabilities are in gray. The rows are previous words and the columns are next words.

Solution:

$$\begin{aligned}
 P(i \text{ want to eat lunch}) &= P(i|\langle s \rangle)P(\text{want}|i)P(\text{to}|\text{want})P(\text{eat}|\text{to})P(\text{lunch}|\text{eat})P(\langle /s \rangle | \text{lunch}) \\
 &= 0.19 \times 0.33 \times 0.66 \times 0.28 \times 0.056 \times 0.4 \\
 &= 0.0025955
 \end{aligned}$$

2. (2 points) Consider a single-headed self-attention mechanism that processes N input tokens of D dimensions to produce N output tokens of the same size. How many weights and biases are used to compute the queries, keys, and values that have D dimensions? What is the size of the attention matrix? If not using attention, how many weights and biases would there be in a fully connected network relating all DN inputs to all DN outputs?

Solution:

- Each of Q, K, V has one weight matrix of size $D \times D$ and a bias for each dimension. Thus, there are $3D + 3D^2$ parameters in total.
- Since the attention score is computed between every pair of tokens, the size of the attention matrix is $N \times N$.
- To connect DN input to DN output, we need $(DN)^2 + DN$ parameters.

3. (2 points) Consider the softmax operation $\mathbf{y} = \text{softmax}(\mathbf{z})$ on a vector $\mathbf{z} = [z_1, z_2, z_3]$.

(a) Given values: $z_1 = -3, z_2 = 1, z_3 = 2$, compute the 9 derivatives, $\frac{\partial y_i}{\partial z_j}$ for all $i, j \in 1, 2, 3$. What do you conclude?

Solution:

If $i = j$:

$$\begin{aligned}\frac{\partial y_i}{\partial z_i} &= \frac{(e^{z_1} + e^{z_2} + e^{z_3}) \cdot e^{z_i} - e^{z_i} \cdot e^{z_i}}{(e^{z_1} + e^{z_2} + e^{z_3})^2} = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + e^{z_3}} - \frac{(e^{z_i})^2}{(e^{z_1} + e^{z_2} + e^{z_3})^2} \\ &= y_i - y_i^2 = y_i \cdot (1 - y_i) = y_i \cdot (1 - y_j)\end{aligned}$$

else if $i \neq j$:

$$\begin{aligned}\frac{\partial y_i}{\partial z_j} &= \frac{(e^{z_1} + e^{z_2} + e^{z_3}) \cdot 0 - e^{z_i} \cdot e^{z_j}}{(e^{z_1} + e^{z_2} + e^{z_3})^2} = -\frac{e^{z_i}}{e^{z_1} + e^{z_2} + e^{z_3}} \cdot \frac{e^{z_j}}{e^{z_1} + e^{z_2} + e^{z_3}} \\ &= -y_i \cdot y_j = y_i \cdot (0 - y_j)\end{aligned}$$

We can conclude that:

$$\frac{\partial y_i}{\partial z_j} = y_i \cdot (\delta_{ij} - y_j)$$

, where $\delta_{ij} = [i = j]$ called Kronecker delta.

We first calculate $y_1 \approx 0.004902, y_2 \approx 0.267623, y_3 \approx 0.727475$.

Then derive the answer matrix:

$$J = \begin{bmatrix} \frac{\partial y_1}{\partial z_1} & \frac{\partial y_1}{\partial z_2} & \frac{\partial y_1}{\partial z_3} \\ \frac{\partial y_2}{\partial z_1} & \frac{\partial y_2}{\partial z_2} & \frac{\partial y_2}{\partial z_3} \\ \frac{\partial y_3}{\partial z_1} & \frac{\partial y_3}{\partial z_2} & \frac{\partial y_3}{\partial z_3} \end{bmatrix} = \begin{bmatrix} 0.004877 & -0.001311 & -0.003565 \\ -0.001311 & 0.196001 & -0.194689 \\ -0.003565 & -0.194689 & 0.198255 \end{bmatrix}$$

Observation:

- It's a symmetric matrix. Diagonal entries are positive, other are negative.
- Each row sums to zero, that is $y_1 - y_1^2 - y_1 y_2 - y_1 y_3 = 0$, which further indicate $y_1 + y_2 + y_3 = 1$.

- (b) Does the output change due to a scaling of the input, i.e. $\text{softmax}(c\mathbf{z})$ for all non-zero c ?

Solution:

For larger $|c|$, the output $\mathbf{y}$ become more sparse, it will scale the original largest become more large, and the smaller one become smaller. (Since the scaling will be exponential). The order change if $c < 0$.

- (c) Does the output change due to a shift of the input, i.e. $\text{softmax}([z_1+c, z_2+c, z_3+c])$ for all real c ?

Solution:

No, as the c will actually be eliminate:

$$\frac{e^{z_1+c}}{e^{z_1+c} + e^{z_2+c} + e^{z_3+c}} = \frac{e^{z_1}e^c}{e^{z_1}e^c + e^{z_2}e^c + e^{z_3}e^c} = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

4. (1 point) Which of the following are true about recurrent neural networks? Select all that apply.
- (a) Training recurrent neural networks can be impeded by the exploding gradient problem
 - (b) Unlike standard feedforward networks, recurrent neural networks can learn from sequences of variable length.
 - (c) Gradient clipping might help if your RNN is troubled by vanishing gradients.
 - (d) None of the above

Solution:

(a), (b)

5. (2 points) This question is about low-rank adaptation (LoRA). Consider a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ in a Transformer model. In LoRA, the fine-tuned weight is parameterized as

$$W = W_0 + \Delta W, \quad \Delta W = BA,$$

where $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, and $r \ll \min(d, k)$.

- (a) Derive the number of trainable parameters in LoRA in terms of d , k , and r . Compare this with the number of trainable parameters in full fine-tuning.

Solution:

- LoRA: $rk + rd = r(k + d)$
- Full: kd

Since r is much smaller, the number of trainable parameters in LoRA is much smaller.

- (b) What is the advantage of LoRA(low-rank adaptation) over adaptor module for parameter-efficient fine-tuning of large language model?

Solution:

LoRA can be applied without changing the model architecture.

6. (2 points) Sampling from distributions.

Recall the reparameterization trick for VAEs. We modeled the distribution of latent z as a Normal Distribution with mean μ and standard deviation σ . Then, instead of directly sampling from z , we sample from $\epsilon \sim \mathcal{N}(0, I)$ and calculate $z = \mu + \sigma\epsilon$.

- (a) What is the purpose of such reparameterization trick?

Solution:

We can't perform backpropagation if we directly sample $z \sim \mathcal{N}(\mu, \sigma^2)$, since it's not differentiable.

- (b) Now instead of modeling the distribution of z values as a normal distribution, you want to model it as a uniform distribution between a and b . Can you perform a similar reparameterization trick in this setting? If yes, give the specific formula of how to do this. If not, explain why not.

Solution:

We first sample $\epsilon \sim \text{Uniform}(0, 1)$, where the mean and standard deviation of distribution of ϵ are $\mu_\epsilon = \frac{1}{2}$ and $\sigma_\epsilon = \frac{1}{\sqrt{12}}$.

Now, we want the reparameterization trick to make the new distribution of $z = f(\epsilon)$ has $\mu_z = \frac{a+b}{2}$ and $\sigma_z = \frac{b-a}{\sqrt{12}}$.

We can take $f(\epsilon) = a + (b - a)\epsilon$, so that

$$\mu_z = a + (b - a)\mu_\epsilon = \frac{a + b}{2}$$

and

$$\sigma_z = (b - a)\sigma_\epsilon = \frac{b - a}{\sqrt{12}}$$